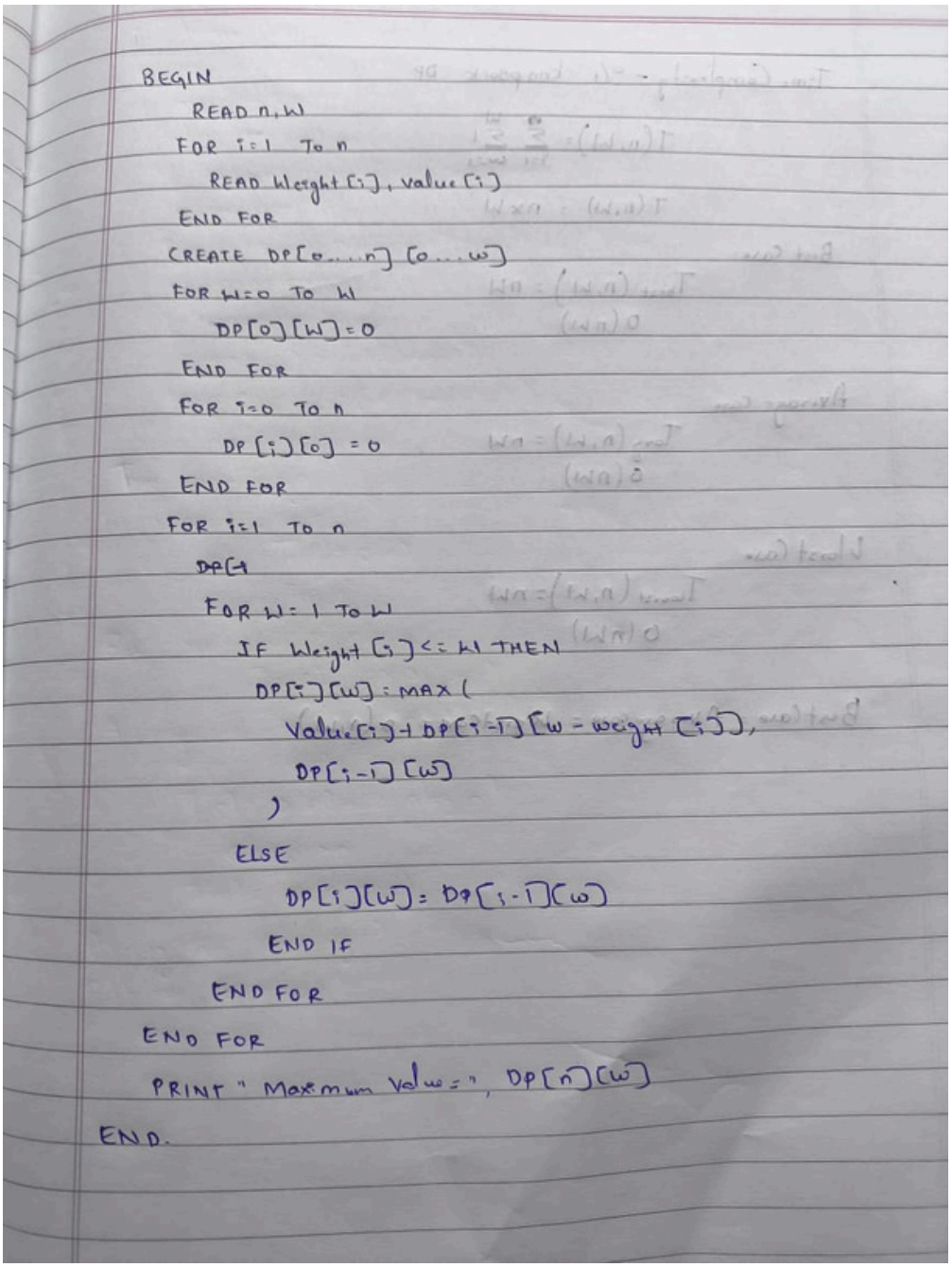
Practical-5

***Aim**: Implementation of a knapsack problem usingdynamic programming.*

Algorithm:



```
BEGIN
  READ n, W
  FOR i=1 To n
    READ Weight[i], value[i]
  END FOR
  CREATE DP[0...n][0...W]
  FOR W=0 To W
    DP[0][W]=0
  END FOR
  FOR i=0 To n
    DP[i][0] = 0
  END FOR
  FOR i=1 To n
   DP[
   FOR W= 1 To W
    IF Weight[i]<=W THEN
     DP[i][W] = MAX(
       Value[i]+DP[i-1][W-Weight[i]],
       DP[i-1][W]
     )
    ELSE
      DP[i][W]= DP[i-1][W]
    END IF
  END FOR
 END FOR
 PRINT " Maximum Value=", DP[n][W]
END.
```

program code:

```
#include<iostream>
using namespace std;

class Knapsack {

private:
   int w[10];       // weights
   int v[10];       // values
   int n;           // number of items
   int W;           // capacity
   int dp[10][100]; // dp table

public:

   //──GetInput──
   void getInput() {
      cout<<"Enternumber of items : ";
      cin >>n;
      cout<<"Enterknapsack capacity : ";
      cin >>W;
      for(inti=1;i<= n; i++) {
         cout<<"Item " << i << " weight and value : ";
         cin>>w[i]>> v[i];
      }
   }

   //──Builddptable ──
   void solve() {
```

```
    //basecase—row 0 and col 0
    for(inti=0;i<=n; i++) dp[i][0] = 0;
    for(intj=0;j<=W; j++) dp[0][j] = 0;

    // fill table
    for(inti=1;i<=n; i++) {
        for(intj=1;j<= W; j++) {

            if(w[i]<=j){
                //takeornot take
                dp[i][j]=max(
                    v[i]+dp[i-1][j-w[i]],
                    dp[i-1][j]
                );
            } else {
                //cannottake   item   i
                dp[i][j]=dp[i-1][j];
            }
        }
    }
}

//——Printdptable——
void printTable() {
    cout<<"\nDPTable:" << endl;
    cout<<"       ";
    for(intj=0;j<=W; j++)
        cout<<j<<"";
    cout << endl;
```

```
    for(int i = 0; i <= n; i++) {
        cout << "i=" << i << " ";
        for(int j = 0; j <= W; j++)
            cout << dp[i][j] << " ";
        cout << endl;
    }
}

// ── Find selected items ──
void findItems() {
    cout<<"\nSelectedItems:"<<endl;
    int j = W;
    for(int i = n; i >= 1; i--) {
        if(dp[i][j] != dp[i-1][j]) {
            cout << "Item " << i
                <<"(w="<<w[i]
                <<",v="<<v[i]
                << ") TAKEN " << endl;
            j = j - w[i];
        } else {
            cout << "Item " << i
                <<"(w="<<w[i]
                <<",v="<<v[i]
                <<")NOTtaken"<<endl;
        }
    }
}

// ── Show result ──
void showResult() {
    cout << "\nMaximum Value = "
```

```
            <<dp[n][W] << endl;
    }
};

//──MAIN──
int main() {

    Knapsackk;

    k.getInput();
    k.solve();
    k.printTable();
    k.showResult();
    k.findItems();

    return 0;
}
```

Output:

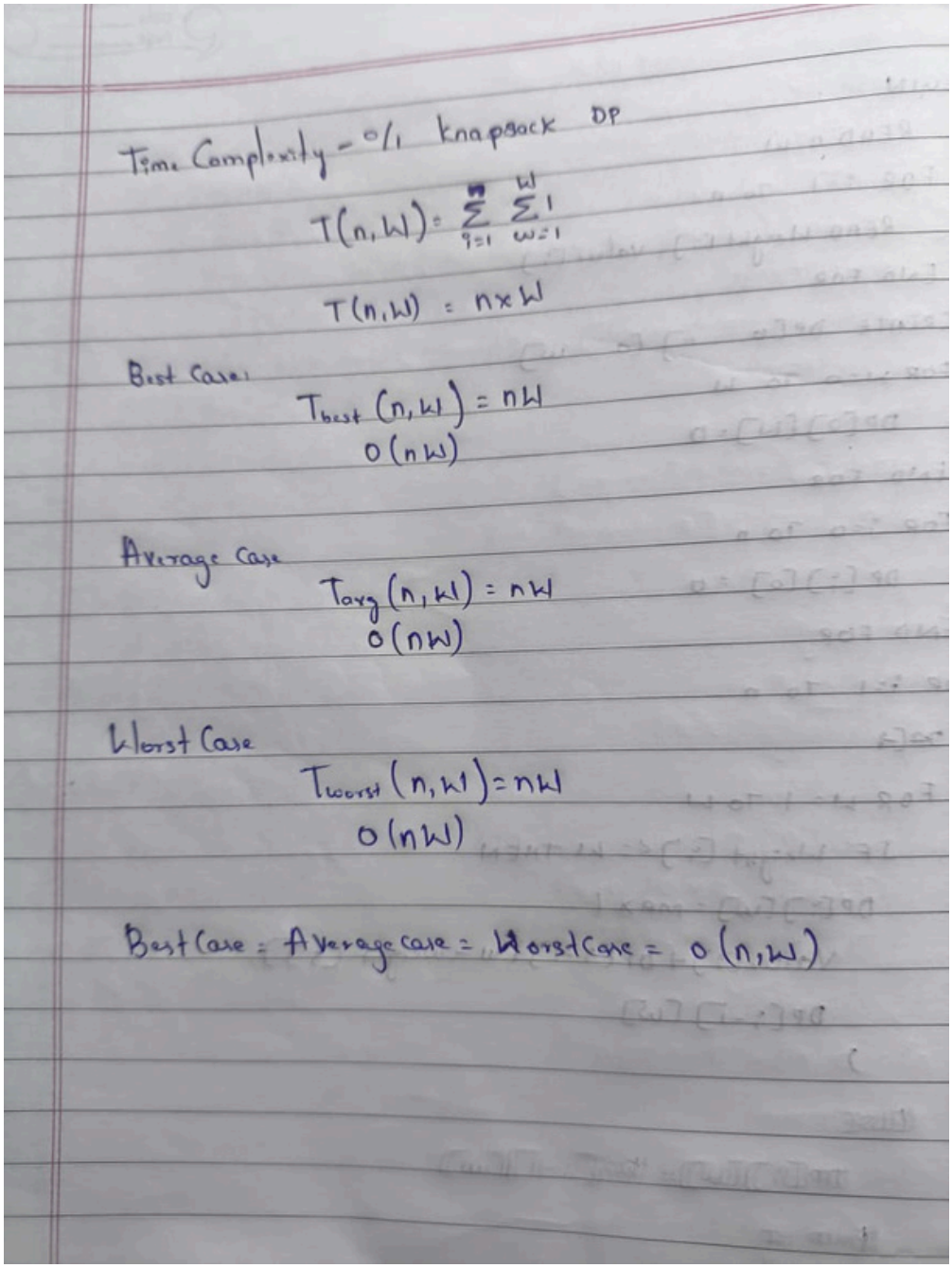
```
PS C:\Users\THARUN\OneDrive\Documents\DAA(practicals)> cd "c:\Users\THARUN\OneDrive\Documents\DAA(practicals)\" ; if ($?) { g++ prac5.cpp -o prac5 } ; if ($?) { .\prac5 }
Enter number of items   : 4
Enter knapsack capacity : 7
Item 1 weight and value : 1 1
Item 2 weight and value : 3 4
Item 3 weight and value : 4 5
Item 4 weight and value : 5 7

DP Table:
     0  1  2  3  4  5  6  7
i=0  0  0  0  0  0  0  0  0
i=1  0  1  1  1  1  1  1  1
i=2  0  1  1  4  5  5  5  5
i=3  0  1  1  4  5  6  6  9
i=4  0  1  1  4  5  7  8  9

Maximum Value = 9

Selected Items:
Item 4 (w=5, v=7) NOT taken
Item 3 (w=4, v=5) TAKEN
Item 2 (w=3, v=4) TAKEN
Item 1 (w=1, v=1) NOT taken
PS C:\Users\THARUN\OneDrive\Documents\DAA(practicals)>
```

Time complexity:



Conclusion:

Thus, the 0/1 Knapsackproblem was successfullyimplemented using Dynamic Programming. The DP table was usedto find the maximum possible value without exceeding the given knapsack capacity. The selected itemswere also identified usingthe completed DP table. The time complexity of the algorithm is O(nW).